Data Structures and Algorithms -3 (25CS2103E)

LCP Array Construction from Suffix Arrays

Team Members

2520030596 - Shivasai

2520030526 - Tarun V

2520030513 - K. Rohan

Under the Esteemed Guidance of

Name of the DSA-3 faculty name with designation and department

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

A suffix array combined with its Longest Common Prefix (LCP) array gives a compact, linear-space alternative to a suffix tree for a wide range of string-matching tasks. This project implements suffix array construction, a naive $O(n^2)$ LCP algorithm, and Kasai's linear-time O(n) LCP algorithm, then builds substring-query support — pattern search, longest repeated substring, distinct-substring counting, and longest common substring — on top of them. Benchmarking on random and worst-case inputs shows Kasai's algorithm outperforming the naive approach by up to several hundred times, confirming its value for large-scale text analytics and compression pipelines such as the Burrows-Wheeler Transform.

Introduction

- A suffix array stores the starting indices of every suffix of a string, sorted lexicographically, and supports fast pattern search via binary search.
- On its own, a suffix array hides how similar neighbouring suffixes are — the LCP array fills this gap by recording shared-prefix lengths between adjacent suffixes.
- Together, SA + LCP arrays emulate most of the functionality of a suffix tree while using far less memory, which matters for large genomic or web-scale text corpora.
- This project builds both structures from scratch in Python, compares a naive $O(n^2)$ LCP algorithm against Kasai's O(n) algorithm, and layers four substring-query operations on top.

Literature Survey

- Manber & Myers (1993) introduced the suffix array as an O(n log n)-buildable, space-efficient alternative to suffix trees for pattern matching.
- Kasai, Lee, Arimura, Arikawa & Park (2001) proposed the linear-time LCP algorithm used in this project, exploiting the fact that the match length can drop by at most one per step.
- Kärkkäinen & Sanders (2003) presented the DC3/skew algorithm, achieving O(n) suffix array construction suitable for very large texts.
- Burrows & Wheeler (1994) built a reversible text transform on sorted rotations equivalent to a suffix array, the basis of bzip2-style compressors.
- Ferragina & Manzini (2000) combined the BWT with suffix arrays to build FM-indexes, enabling compressed full-text search.

Problem Statement and Objectives

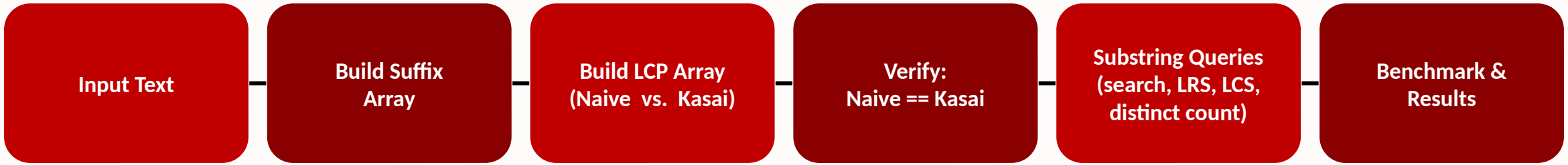
Problem Statement:

Computing the LCP array by brute force takes $O(n^2)$ time in the worst case, which becomes impractical for large texts used in compression and analytics pipelines. There is a need for a linear-time construction method, together with a clear demonstration of its practical benefit on both typical and pathological inputs.

Objectives:

- Implement suffix array construction and a naive $O(n^2)$ LCP algorithm as a correctness baseline.
- Implement Kasai's O(n) algorithm and verify it produces identical results to the naive method.
- Build substring-query operations — pattern search, longest repeated substring, distinct substrings, longest common substring — on top of the SA + LCP arrays.
- Benchmark both LCP methods on random and worst-case text to quantify the real-world speed-up.

Flow Chart



The pipeline builds one suffix array, then computes the LCP array two ways for cross-validation. Once verified, the same arrays power every substring query, and timings from both LCP methods feed directly into the benchmark results.

Results

Benchmark: Random text (4-letter alphabet)

n	SA build (s)	Naive LCP (s)	Kasai LCP (s)	Speed-up
100	0.00017	0.00003	0.00002	1.6x
500	0.00096	0.00028	0.00013	2.1x
1000	0.00240	0.00060	0.00028	2.2x
2000	0.00500	0.00132	0.00057	2.3x
4000	0.01212	0.00298	0.00117	2.6x

Worst case: text = "a" × n (naive LCP degrades toward $O(n^2)$)

n	Naive LCP (s)	Kasai LCP (s)	Speed-up
200	0.00145	0.00003	47.6x
800	0.03609	0.00017	215.0x
1600	0.15313	0.00039	389.1x
3200	0.61765	0.00079	786.0x

Up to 786x faster

at n = 3200, and correctness verified against the naive method every run

Hardware and Software requirements

Hardware Requirements:

- Standard PC / laptop, dual-core processor or better
- Minimum 4 GB RAM (8 GB recommended for larger benchmark sizes)
- No GPU required — all algorithms run on the CPU

Software Requirements:

- Python 3.10 or later (reference implementation uses only the standard library)
- Optional: matplotlib, for plotting benchmark charts
- Any code editor / IDE (VS Code, PyCharm, Jupyter, etc.)
- Operating System: Windows, Linux, or macOS — fully platform independent

Conclusion

- Kasai's algorithm computes the LCP array in O(n), verified against the O(n^2) naive method for correctness on every test case.
- Benchmarks confirm the theoretical gap: up to ~786x faster on highly repetitive worst-case text, and 2–3x faster even on typical random text.
- The combined suffix array + LCP array structure answers substring queries — pattern search, longest repeated substring, distinct-substring counts, and longest common substrings — efficiently and with modest memory.
- These structures directly underpin real systems: BWT-based compressors, plagiarism / duplicate-content detection, and genome alignment tools.

Future Scope

- Extend suffix array construction to the O(n) DC3/skew algorithm for handling very large texts such as genomic sequences or web-scale corpora.
- Build a full suffix-tree emulation layer (e.g. LCP-interval trees) on top of the SA + LCP arrays for richer structural queries.
- Apply the toolkit inside a real Burrows-Wheeler Transform compressor and measure end-to-end compression ratio and speed.
- Parallelize LCP construction and pattern search for multi-core or distributed processing of large datasets.

(DEEMED TO BE UNIVERSITY)

Thank you